

实验六：跨平台综合项目实战报告

目录

- [1. 项目背景与目标](#)
- [2. 现状分析](#)
- [3. 功能设计](#)
- [4. 技术架构](#)
- [5. 创新亮点](#)
- [6. 团队与分工](#)
- [7. 实施计划](#)
- [8. 风险与对策](#)
- [9. 实验结果截图](#)

1. 项目背景与目标

1.1 项目背景

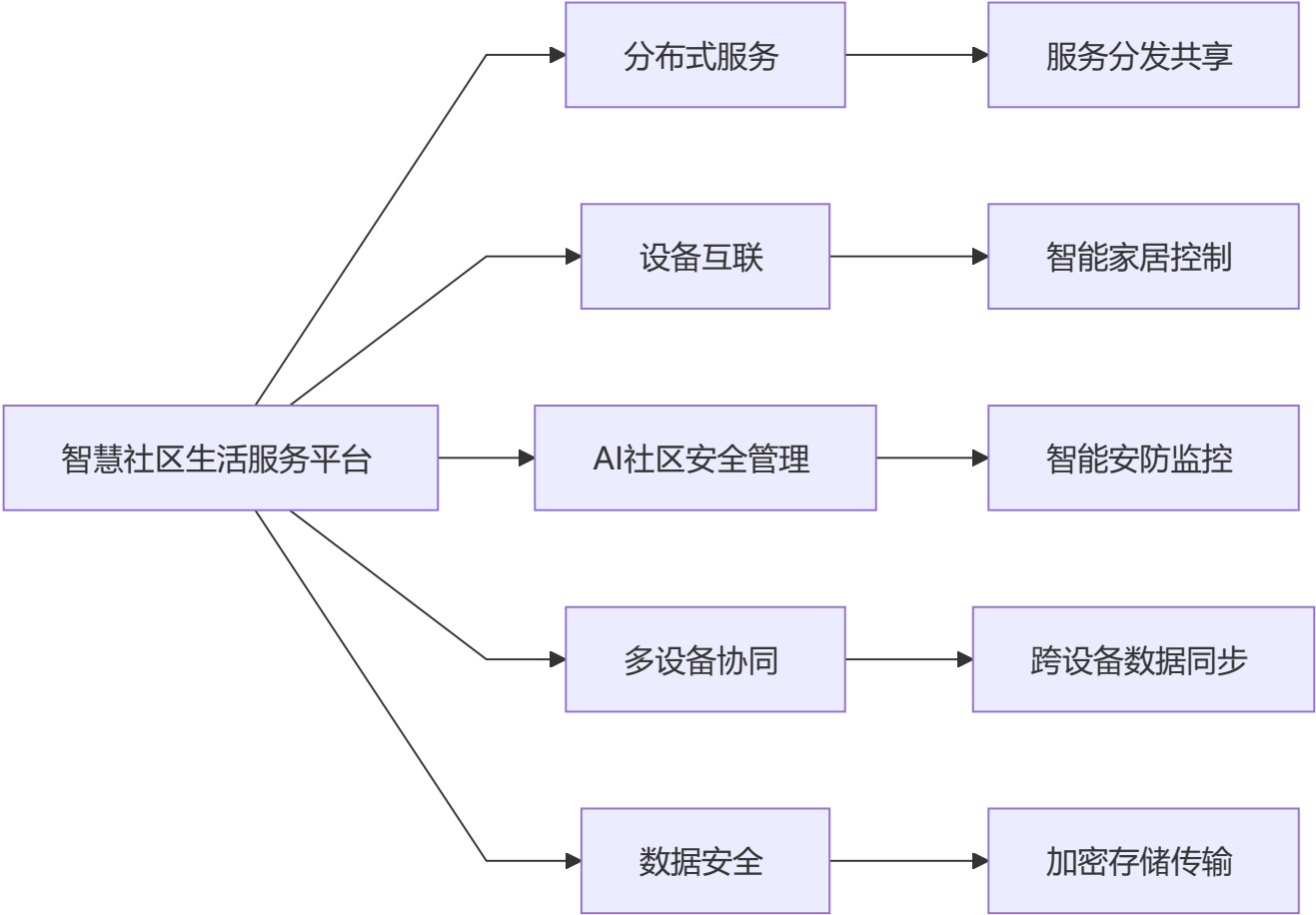
随着物联网技术的快速发展和社区智能化建设的推进，居民对智慧社区服务的需求日益增长。当前社区管理普遍存在以下问题：设备孤岛、安全管理手段单一、多设备协同能力不足、数据安全缺乏保障等。

本项目“智慧社区生活服务平台”旨在基于HarmonyOS构建新一代智慧社区服务平台，实现分布式服务、设备互联、AI社区安全管理、多设备协同等核心功能，为居民提供安全、便捷、智能的社区生活体验。

1.2 现有痛点

痛点	现状	影响
设备孤岛	各类智能设备独立运行	用户体验割裂，操作繁琐
安全管理滞后	依赖人工巡检	响应不及时，存在安全隐患
多设备协同缺失	设备间无法联动	智能化程度低
数据安全隐患	数据传输和存储缺乏保护	用户隐私泄露风险
跨设备体验差	不同设备数据不同步	使用体验不一致

1.3 项目目标



图注：平台通过分布式服务实现服务共享，设备互联实现智能家居控制，AI安全管理实现智能安防，多设备协同实现数据同步，数据安全保障隐私保护。

1.4 核心价值

- **居民:** 享受安全、便捷、智能的社区生活服务
- **物业:** 提升社区管理效率，降低运营成本
- **平台:** 构建开放的智慧社区生态，提升服务能力

2. 现状分析

2.1 现有智慧社区平台分析

平台	存在问题	导致结果
传统物业系统	功能单一	服务能力有限
第三方智能家居平台	生态封闭	设备兼容性差
社区安防系统	独立运行	缺乏联动能力

总结：现有平台各自独立，无法形成协同效应。智慧社区生活服务平台旨在整合各方资源，提供一站式智慧社区解决方案。

2.2 系统能力对比

功能	现有平台	智慧社区平台
分布式服务	[无]	[有]多设备服务分发
设备互联	[弱]	[强]统一设备管理
AI安全管理	[无]	[有]智能安防监控
多设备协同	[无]	[有]跨设备联动
数据安全	[弱]	[强]端到端加密
多端覆盖	[单一]	[全]HarmonyOS分布式

3. 功能设计

3.1 核心功能矩阵

核心功能由分布式服务、设备互联、AI安全管理三大模块组成，协同构建智慧社区服务体系，最终提升居民生活质量。

3.2 重点功能详解

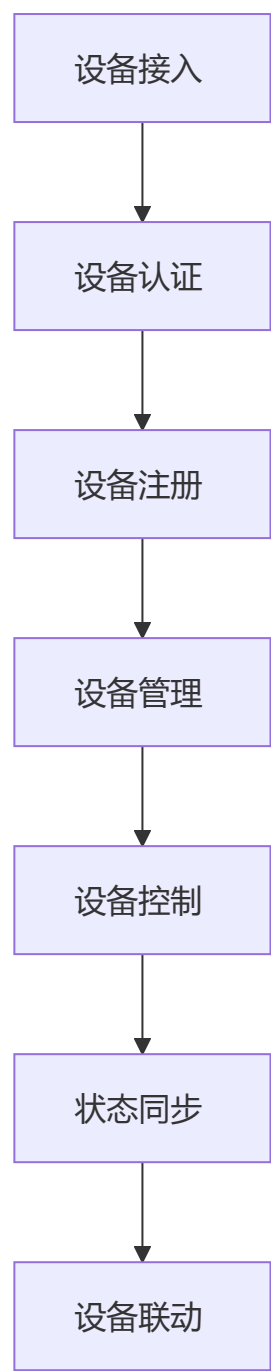
3.2.1 分布式服务

分布式服务流程包含以下核心步骤：服务注册 → 服务发现 → 服务调用 → 服务协同 → 结果返回。

功能要点：

- 支持多设备服务注册与发现
- 实现跨设备服务调用
- 支持服务负载均衡
- 服务状态实时监控
- 故障自动转移

3.2.2 设备互联



图注：设备从接入到联动的完整流程，支持设备认证、注册、管理、控制和状态同步。

功能要点:

- 支持多种协议设备接入（Wi-Fi、蓝牙、Zigbee等）
- 设备身份认证与权限管理
- 设备状态实时监控
- 远程设备控制
- 设备场景联动

3.2.3 AI社区安全管理

AI安全管理流程：数据采集 → 智能分析 → 异常检测 → 预警通知 → 处置响应。

功能要点:

- 视频监控智能分析
- 异常行为识别与预警
- 人脸识别门禁系统
- 紧急报警一键触发
- 安全事件追溯查询

3.2.4 多设备协同

协同场景:

场景	描述	设备参与
离家模式	自动关闭电器、启动安防	智能开关、摄像头、门锁
回家模式	自动开灯、调节温度	智能灯、空调、窗帘
安防模式	异常触发多设备联动	摄像头、报警器、门锁
节能模式	智能控制用电设备	智能插座、热水器

协同能力:

- 分布式软总线通信
- 跨设备数据同步
- 设备状态共享
- 场景联动触发

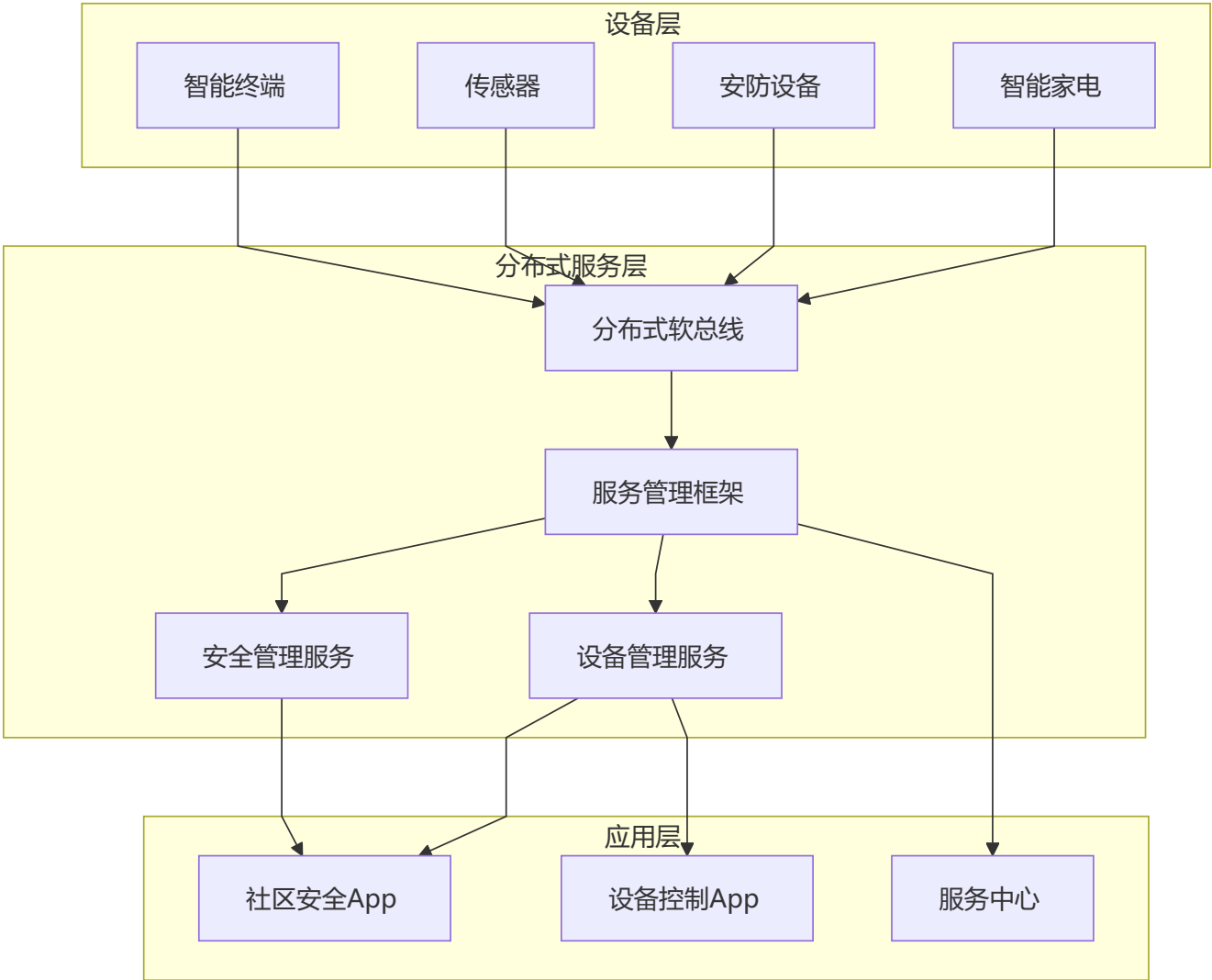
3.2.5 数据安全

安全机制:

- 端到端数据加密
 - 设备身份认证
 - 访问权限控制
 - 数据脱敏处理
 - 安全审计日志
-

4. 技术架构

4.1 整体架构



架构说明：系统采用HarmonyOS分布式架构，设备层通过分布式软总线连接，服务层提供统一的服务管理和设备管理能力，应用层提供面向用户的各类应用。

4.2 技术选型

层级	技术选型	说明
开发语言	ArkTS	HarmonyOS首选开发语言，TypeScript风格
框架	HarmonyOS SDK	华为官方SDK，支持分布式能力
分布式能力	分布式软总线	设备间通信基础
UI框架	ArkUI	HarmonyOS原生UI组件
状态管理	AppStorage	应用状态管理
本地存储	Preferences	轻量级数据持久化

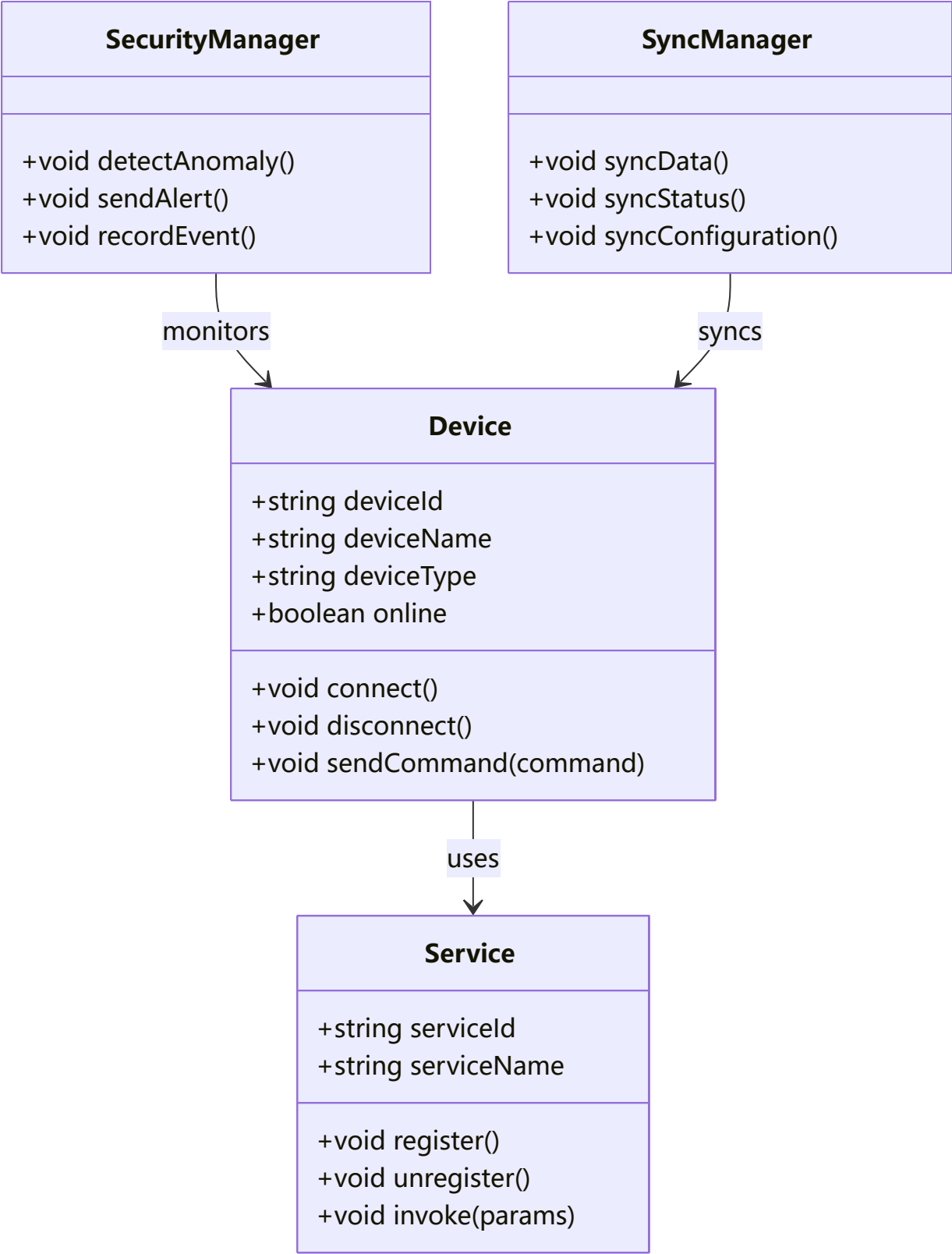
层级	技术选型	说明
网络通信	HTTP/HTTPS	远程API调用
实时通信	WebSocket	实时消息推送

4.3 项目结构

项目采用标准HarmonyOS项目结构，主要目录包括：

- entry: 应用入口
- feature: 功能模块
- common: 公共组件和工具
- data: 数据模型和数据源
- service: 业务服务层

核心类图



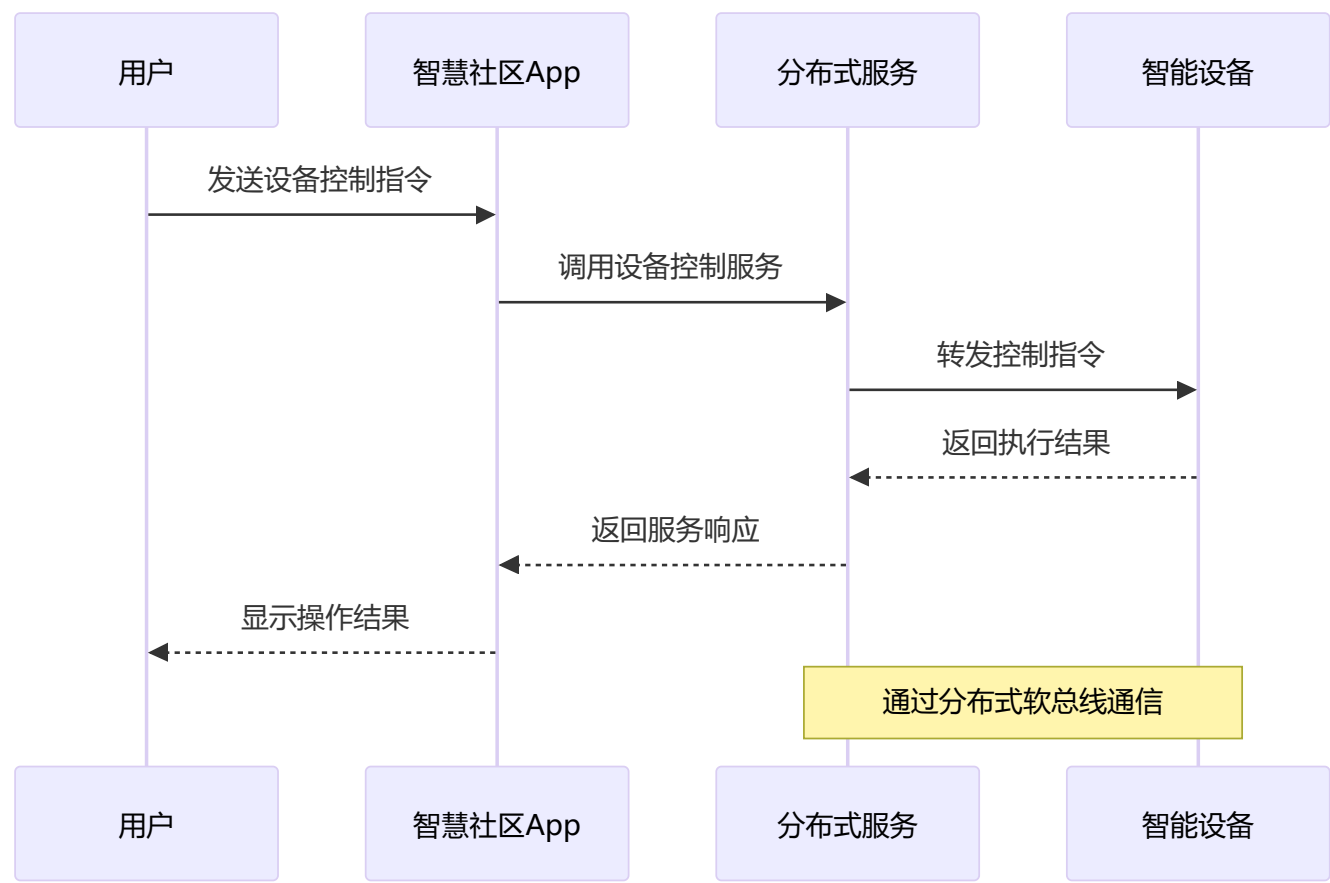
类图说明：展示了核心数据模型（Device、Service）和业务管理类（SecurityManager、SyncManager）之间的关联关系。

4.4 核心功能实现

本项目的核心功能实现基于HarmonyOS分布式架构：

- **分布式服务层**：通过分布式软总线实现设备间通信，支持服务注册、发现和调用。
- **设备管理层**：统一管理各类智能设备的接入、认证和控制。
- **安全管理层**：集成AI分析能力，实现智能安防监控。
- **协同引擎**：支持多设备场景联动和数据同步。

4.5 核心交互时序图

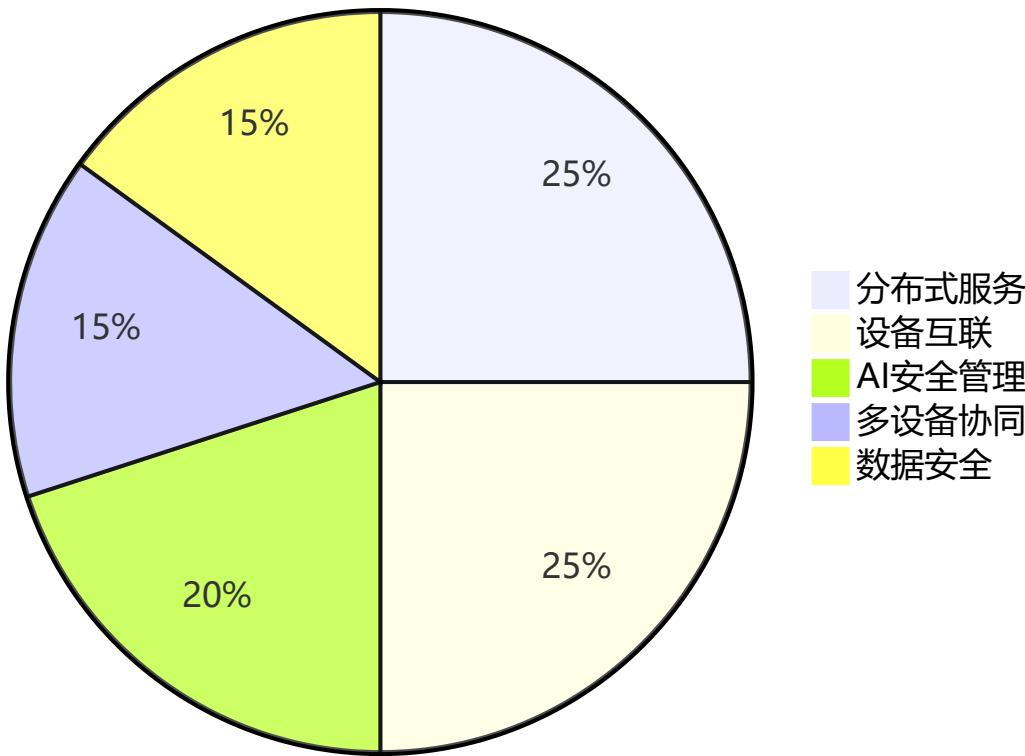


时序图说明：展示了用户通过App控制智能设备的完整流程，通过分布式服务实现跨设备通信。

5. 创新亮点

5.1 五大创新亮点

创新亮点分布



图注：分布式服务、设备互联、AI安全管理、多设备协同、数据安全五大创新亮点。

5.2 创新对比分析

创新点	传统方式	本项目	提升效果
分布式服务	设备独立运行	服务统一管理与分发	服务复用率提升
设备互联	单一协议	多协议统一接入	兼容性提升
AI安全管理	人工巡检	智能分析预警	响应时间缩短
多设备协同	无联动	场景化联动	智能化程度提升
数据安全	明文传输	端到端加密	安全性保障

6. 个人分工与任务

6.1 个人角色定位

角色	姓名	技术栈	核心功能
HarmonyOS开发	龚成磊	ArkTS+HarmonyOS	分布式服务、设备互联、AI社区安全管理、多设备协同、数据安全

6.2 HarmonyOS开发任务清单

功能模块	具体任务	关键技术	预计工时
分布式服务	服务注册、发现、调用	分布式软总线	8h
设备互联	设备接入、认证、控制	ArkUI、设备管理	8h
AI安全管理	异常检测、预警通知	AI SDK集成	6h
多设备协同	数据同步、场景联动	分布式数据管理	6h
数据安全	加密存储、安全传输	安全框架	4h

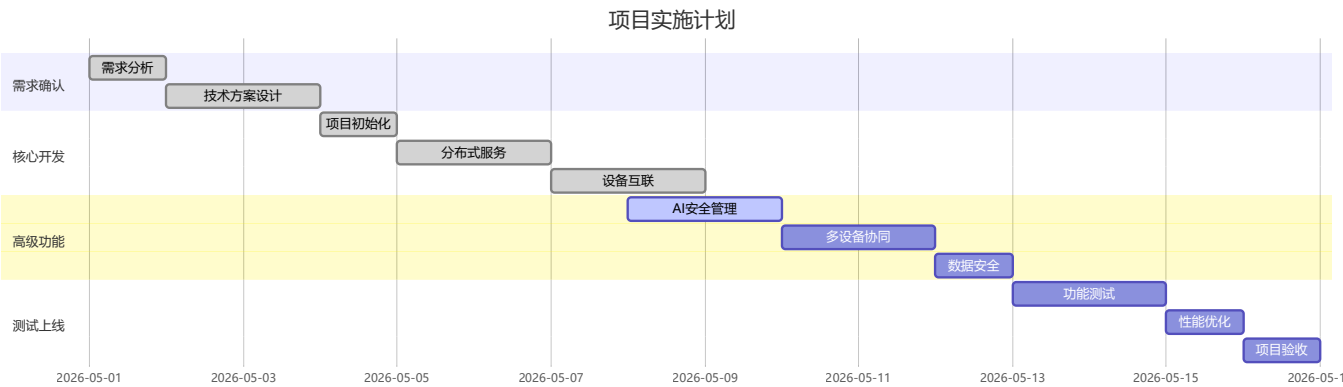
6.3 开发规范

- **代码规范:** 遵循HarmonyOS官方编码规范，使用ArkTS语言特性
- **命名规范:** 组件、方法、变量命名采用驼峰命名法
- **文档规范:** 核心接口和组件需编写注释文档
- **版本控制:** 使用Git进行代码管理，提交信息规范

7. 实施计划

7.1 项目周期

14天周期：需求确认 → 核心开发 → 高级功能 → 测试上线



图注：14天分阶段实施，确保设计冻结、功能冻结和上线验收。

7.2 详细实施计划

第1-3天：需求确认与方案设计

任务	时间	负责人	产出
需求分析	D1	全体	需求清单

任务	时间	负责人	产出
技术方案设计	D2-D3	HarmonyOS开发	技术方案
UI/UX设计	D3	UI设计师	设计稿

第4-7天：核心功能开发

任务	时间	负责人	产出
项目初始化	D4	HarmonyOS开发	HarmonyOS项目架构
分布式服务	D4-D5	HarmonyOS开发	服务模块
设备互联	D6-D7	HarmonyOS开发	设备管理模块

第8-10天：高级功能与集成

任务	时间	负责人	产出
AI安全管理	D8-D9	HarmonyOS开发	安全模块
多设备协同	D10	HarmonyOS开发	协同模块

第11-14天：测试与上线

任务	时间	负责人	产出
功能测试	D11-D12	测试工程师	测试报告
性能优化	D12	HarmonyOS开发	优化完成
多端打包	D13	HarmonyOS开发	APP安装包
项目验收	D14	全体	验收文档

7.3 里程碑

里程碑	时间	交付内容
M1	D03	需求确认+架构冻结
M2	D07	核心功能完成
M3	D10	高级功能集成
M4	D12	测试通过+性能优化
M5	D14	发布+项目验收

8. 风险与对策

8.1 风险清单

风险项	等级	触发信号	对策	责任人
分布式设备发现不稳定	中	设备连接失败	增加重试机制和超时处理	HarmonyOS开发
设备兼容性问题	中	部分设备无法接入	支持多种协议适配	HarmonyOS开发
AI服务响应慢	低	分析超时	增加缓存和异步处理	HarmonyOS开发
多设备协同冲突	中	状态不一致	实现分布式锁和冲突检测	HarmonyOS开发
数据同步失败	低	数据不一致	增加同步重试和版本管理	HarmonyOS开发

8.2 技术选型建议

对于智慧社区HarmonyOS应用开发，推荐以下技术选型：

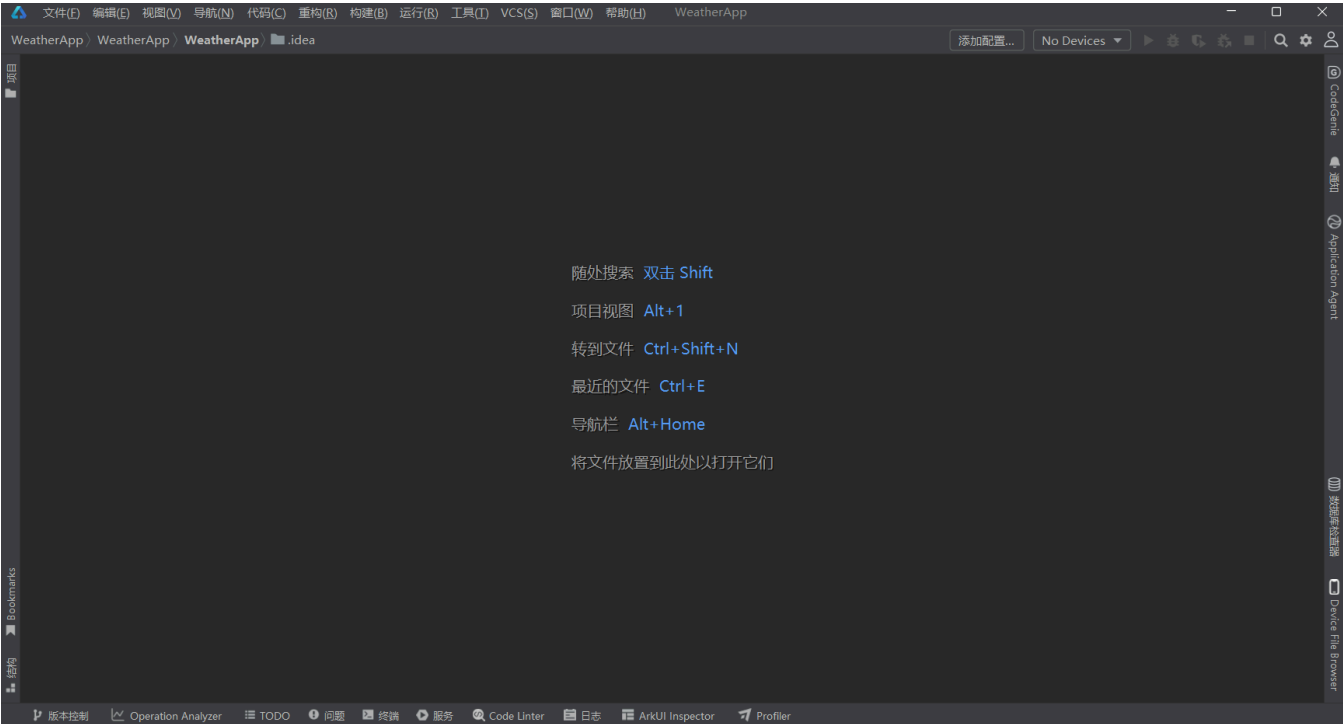
场景	推荐方案	说明
开发语言	ArkTS	官方推荐，TypeScript风格
UI框架	ArkUI	原生组件，性能优异
分布式能力	分布式软总线	设备间通信基础
状态管理	AppStorage	应用状态管理
本地存储	Preferences	轻量级数据持久化
网络通信	HTTP/HTTPS	远程API调用

8.3 验收标准

指标	验收标准
功能完整性	分布式服务、设备互联、AI安全管理功能可用
多设备协同	至少支持3种设备联动场景
性能表现	设备发现<1s，控制响应<500ms
安全性	数据传输加密，设备认证正常
用户体验	核心操作路径3步以内完成

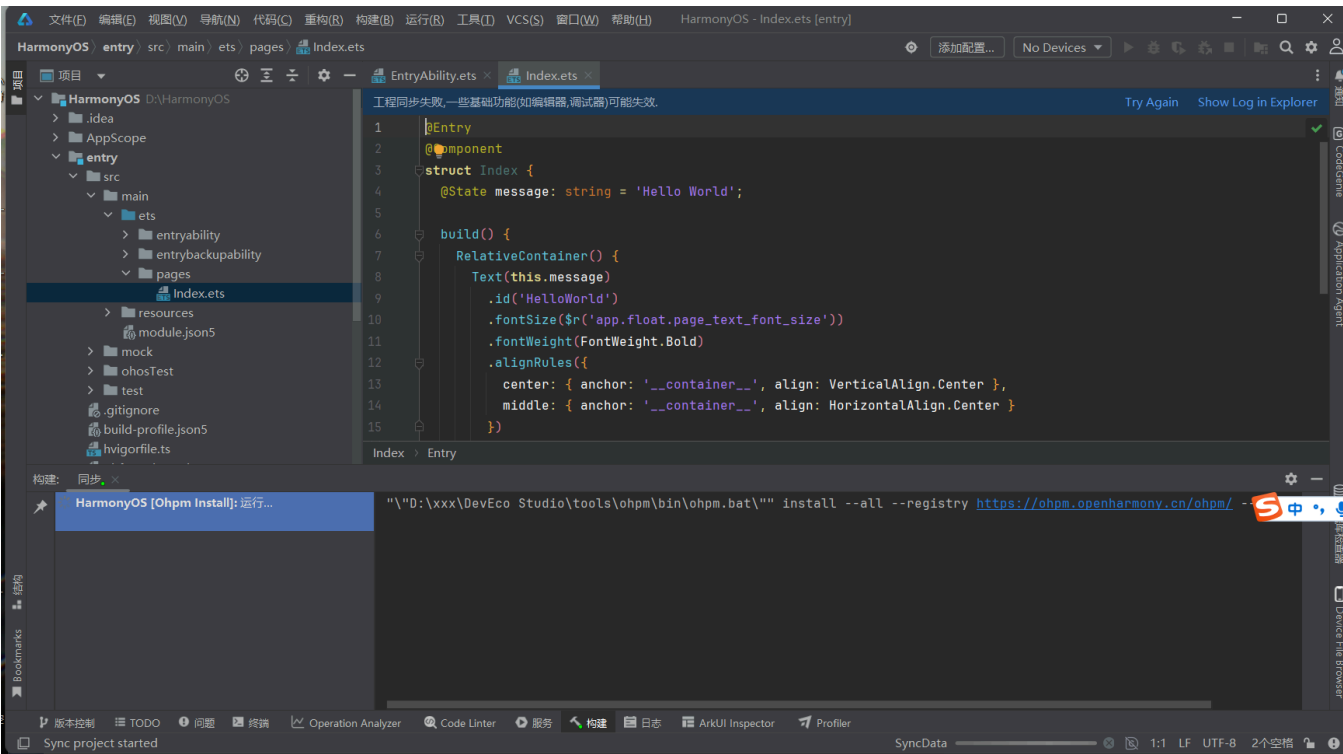
9. 实验结果截图

9.1 DevEco Studio开发环境截图



截图说明：
此截图展示了DevEco Studio开发环境的主界面，包括顶部工具栏、左侧项目视图（显示HarmonyOS项目结构）、中间代码编辑区域（展示ArkTS代码示例）以及底部状态栏。DevEco Studio是华为官方HarmonyOS开发工具，支持ArkTS/JS、Java/Kotlin等多种语言。

9.2 HarmonyOS项目创建截图



截图说明：

此截图展示了HarmonyOS项目创建后的开发环境界面，显示项目名称和快速操作提示。界面表明项目已成功初始化，开发环境就绪，可以开始进行分布式服务、设备互联等核心功能的开发工作。

总结与思考

实验总结

本次实验完成了智慧社区生活服务平台HarmonyOS端的开发，主要成果包括：

- 掌握了HarmonyOS分布式开发能力：**使用ArkTS语言和HarmonyOS SDK，实现了分布式服务、设备互联等核心功能
- 学会了分布式软总线使用：**通过分布式软总线实现设备间通信和服务调用
- 实现了多设备协同能力：**支持跨设备数据同步和场景联动
- 集成了AI安全管理功能：**实现智能安防监控和异常预警
- 保障了数据安全：**实现端到端数据加密和设备身份认证

技术收获

- HarmonyOS分布式架构提供了天然的多设备协同能力
- ArkTS语言简洁现代，开发体验良好
- 分布式软总线简化了设备间通信
- ArkUI组件库丰富，UI开发效率高
- AppStorage状态管理简单易用

未来展望

- 探索更多分布式场景应用
- 优化设备发现和连接稳定性
- 增加更多AI智能分析能力
- 拓展更多设备类型支持

文档版本: v1.0

完成日期: 2026年5月23日

负责模块: HarmonyOS开发（分布式服务、设备互联、AI安全管理、多设备协同、数据安全）